Report No: 1
Team: MACH::GO
Date: 21 December 2025

1. Introduction

This report summarises the initial progress of the team for the December reporting cycle. The main achievements include finalizing the hardware architecture with a migration to Ubuntu 24.04, establishing stable low-level control via UART, and completing the car's kinematic URDF model for simulation.

2. Planned activities

At the beginning of this reporting period, several activities were planned to set up the project for upcoming development stages. The planned activities were as follows:

Hardware Configuration Planning: This included identifying the most suitable computing platform and determining optimal sensor placement on the car. While the current setup is based on a Raspberry Pi, the system is designed with a future migration to an NVIDIA Jetson Orin Nano in mind.

Software Architecture Planning: Defining the overall ROS2 node structure, including the communication strategy between the high-level controller running on the Raspberry Pi and the low-level controller implemented on the Nucleo board.

Collaboration Workflow Setup: Creating a GitHub organisation and defining branching and version control practices to support collaborative development across the team.

Project Planning: Establishing a project timeline and assigning responsibilities to ensure that team members are aligned with upcoming milestones.

3. Status of planned activities

A. Environment Setup & OS Migration

Status: Completed

Implementation: The original plan was to use Ubuntu 22.04 LTS with ROS2 Humble, as this is a common configuration for ROS based systems. However, during initial setup it became clear that the Raspberry Pi 5 does not officially support Ubuntu 22.04. In order to take advantage of the improved processing capabilities of the Pi 5 without compromising system compatibility, the team migrated to Ubuntu 24.04. ROS2 was then built from source to ensure full functionality on the new operating system.

Difficulties: Building ROS2 from source required resolving several dependency and compatibility issues. While this process was time-consuming, the resulting environment is now stable and better suited to the hardware platform being used.

B. Low-Level Control & Teleoperation

Status: Completed

Implementation: We established serial communication (UART) between the Raspberry Pi and the Nucleo to translate ROS 2 velocity commands into motor PWM signals.

Difficulties: Initial testing showed latency and inconsistent motor behaviour. While the steering system responded correctly, the drive motors were often unresponsive. While looking into it, we found that ROS2 node was publishing velocity commands at a frequency higher than the Nucleo's UART buffer could handle, resulting in buffer overflows and dropped commands.

Solution: To fix this issue, a control rate limiter was implemented on the ROS2 node, limiting command publication to 50 Hz. This change removed the buffer overflow problem, and the car now responds smoothly to teleoperation commands with no noticeable latency.

C. Kinematics & URDF Modeling

Status: Completed

Implementation: A detailed URDF model was created to act as a digital twin of the car. This model will reflect both the kinematic behaviour and the physical structure of the car.

The car's Ackermann steering geometry was made using nested joints, enabling realistic steering visualisation in Rviz. Hard limits were applied to the steering joints to match the physical limits of the servo and prevent potential hardware damage during operation. In addition, a complete static TF tree was defined for the IMU, cameras, and ToF sensors using measured chassis dimensions, including a wheelbase of 0.255 m and a track width of 0.165 m.

4. General status of the project

At its current stage, the project has reached a stable and functional baseline. The car can be driven remotely through teleoperation with a consistent and predictable motor response. The software stack successfully visualises the robot state and sensor placements in Rviz, providing a clear representation of the system configuration.

However, the car is not yet operating autonomously. As for the sensors, they have not been calibrated and are not yet integrated into a perception pipeline. As a result, the system currently relies entirely on manual operator input. Despite this limitation, the implementation of the 50 Hz control rate has improved system stability and ensures a reliable communication between the planner and the actuators.

5. Upcoming activities

During the next reporting period, our development efforts will shift toward enabling perception and simulation based testing. The planned activities include integrating the ordered cameras and sensors with the Raspberry Pi, followed by intrinsic and extrinsic calibration. In parallel, the completed URDF model will be used within a Gazebo simulation environment to test navigation and control algorithms before deploying it on the physical car. Also, a basic perception-to-control pipeline will be implemented to process camera data for lane detection and then we will proceed to build the autonomous behaviour.